JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Interface SortedSet<E>

Type Parameters:

`E - the type of elements maintained by this set`

All Superinterfaces:

`Collection<E>, Iterable<E>, Set<E>`

All Known Subinterfaces:

`NavigableSet<E>`

All Known Implementing Classes:

`ConcurrentSkipListSet, TreeSet`

```
public interface SortedSet<E>
extends Set<E>
```

A `Set` that further provides a *total ordering* on its elements. The elements are ordered using their natural ordering, or by a `Comparator` typically provided at sorted set creation time. The set's iterator will traverse the set in ascending element order. Several additional operations are provided to take advantage of the ordering. (This interface is the set analogue of `SortedMap`.)

All elements inserted into a sorted set must implement the `Comparable` interface (or be accepted by the specified comparator). Furthermore, all such elements must be *mutually comparable*: `e1.compareTo(e2)` (or `comparator.compare(e1, e2)`) must not throw a `ClassCastException` for any elements `e1` and `e2` in the sorted set. Attempts to violate this restriction will cause the offending method or constructor invocation to throw a `ClassCastException`.

Note that the ordering maintained by a sorted set (whether or not an explicit comparator is provided) must be *consistent with equals* if the sorted set is to correctly implement the `Set` interface. (See the `Comparable` interface or `Comparator` interface for a precise definition of *consistent with equals*.) This is so

because the `Set` interface is defined in terms of the `equals` operation, but a sorted set performs all element comparisons using its `compareTo` (or `compare`) method, so two elements that are deemed equal by this method are, from the standpoint of the sorted set, equal. The behavior of a sorted set *is* well-defined even if its ordering is inconsistent with equals; it just fails to obey the general contract of the `Set` interface.

All general-purpose sorted set implementation classes should provide four "standard" constructors: 1) A void (no arguments) constructor, which creates an empty sorted set sorted according to the natural ordering of its elements. 2) A constructor with a single argument of type `Comparator`, which creates an empty sorted set sorted according to the specified comparator. 3) A constructor with a single argument of type `Collection`, which creates a new sorted set with the same elements as its argument, sorted according to the natural ordering of the elements. 4) A constructor with a single argument of type `SortedSet`, which creates a new sorted set with the same elements and the same ordering as the input sorted set. There is no way to enforce this recommendation, as interfaces cannot contain constructors.

Note: several methods return subsets with restricted ranges. Such ranges are *half-open*, that is, they include their low endpoint but not their high endpoint (where applicable). If you need a *closed range* (which includes both endpoints), and the element type allows for calculation of the successor of a given value, merely request the subrange from `lowEndpoint` to `successor(highEndpoint)`. For example, suppose that `s` is a sorted set of strings. The following idiom obtains a view containing all of the strings in `s` from `low` to `high`, inclusive:

```
SortedSet<String> sub = s.subSet(low, high+"\0");
```

A similar technique can be used to generate an *open range* (which contains neither endpoint). The following idiom obtains a view containing all of the Strings in `s` from `low` to `high`, exclusive:

```
SortedSet<String> sub = s.subSet(low+"\0", high);
```

This interface is a member of the Java Collections Framework.

Since:

1.2

See Also:

`Set`, `TreeSet`, `SortedMap`, `Collection`, `Comparable`, `Comparator`, `ClassCastException`

Method Summary

Abstract Methods | **Default Methods**

Modifier and Type	Method	Description

`Comparator<? super E>`	`comparator()`	Returns the comparator used to order the elements in this set, or `null` if this set uses the **natural ordering** of its elements.
`E`	`first()`	Returns the first (lowest) element currently in this set.
`SortedSet<E>`	`headSet(E toElement)`	Returns a view of the portion of this set whose elements are strictly less than `toElement`.
`E`	`last()`	Returns the last (highest) element currently in this set.
`default Spliterator<E>`	`spliterator()`	Creates a `Spliterator` over the elements in this sorted set.
`SortedSet<E>`	`subSet(E fromElement, E toElement)`	Returns a view of the portion of this set whose elements range from `fromElement`, inclusive, to `toElement`, exclusive.
`SortedSet<E>`	`tailSet(E fromElement)`	Returns a view of the portion of this set whose elements are greater than or equal to `fromElement`.

Methods inherited from interface java.util.Set

`add`, `addAll`, `clear`, `contains`, `containsAll`, `equals`, `hashCode`, `isEmpty`, `iterator`, `remove`, `removeAll`, `retainAll`, `size`, `toArray`, `toArray`

Methods inherited from interface java.util.Collection

parallelStream, removeIf, stream

Methods inherited from interface java.lang.Iterable

forEach

Method Detail

comparator

```
Comparator<? super E> comparator()
```

Returns the comparator used to order the elements in this set, or `null` if this set uses the natural ordering of its elements.

Returns:

the comparator used to order the elements in this set, or null if this set uses the natural ordering of its elements

subSet

```
SortedSet<E> subSet(E fromElement,
                    E toElement)
```

Returns a view of the portion of this set whose elements range from `fromElement`, inclusive, to `toElement`, exclusive. (If `fromElement` and `toElement` are equal, the returned set is empty.) The returned set is backed by this set, so changes in the returned set are reflected in this set, and vice-versa. The returned set supports all optional set operations that this set supports.

The returned set will throw an `IllegalArgumentException` on an attempt to insert an element outside its range.

Parameters:

fromElement - low endpoint (inclusive) of the returned set

toElement - high endpoint (exclusive) of the returned set

Returns:

a view of the portion of this set whose elements range from fromElement, inclusive, to toElement, exclusive

Throws:

ClassCastException - if fromElement and toElement cannot be compared to one another using this set's comparator (or, if the set has no comparator, using natural ordering). Implementations may, but are not required to, throw this exception if fromElement or toElement cannot be compared to elements currently in the set.

NullPointerException - if fromElement or toElement is null and this set does not permit null elements

IllegalArgumentException - if fromElement is greater than toElement; or if this set itself has a restricted range, and fromElement or toElement lies outside the bounds of the range

headSet

SortedSet<E> headSet(E toElement)

Returns
a view of the portion of this set whose elements are strictly less than toElement. The returned set is backed by this set, so changes in the returned set are reflected in this set, and vice-versa. The returned set supports all optional set operations that this set supports.

The returned set will throw an IllegalArgumentException on an attempt to insert an element outside its range.

Parameters:

toElement - high endpoint (exclusive) of the returned set

Returns:

a view of the portion of this set whose elements are strictly less than toElement

Throws:

ClassCastException - if toElement is not compatible with this set's comparator (or, if the set has no comparator, if toElement does not implement Comparable). Implementations may, but are not required to, throw this exception if toElement cannot be compared to elements currently in the set.

NullPointerException - if toElement is null and this set does not permit null elements

IllegalArgumentException - if this set itself has a restricted range, and toElement lies outside the bounds of the range

tailSet

SortedSet<E> tailSet(E fromElement)

Returns a view of the portion of this set whose elements are greater than or equal to `fromElement`. The returned set is backed by this set, so changes in the returned set are reflected in this set, and vice-versa. The returned set supports all optional set operations that this set supports.

The returned set will throw an `IllegalArgumentException` on an attempt to insert an element outside its range.

Parameters:

`fromElement - low endpoint (inclusive) of the returned set`

Returns:

`a view of the portion of this set whose elements are greater than or equal to fromElement`

Throws:

`ClassCastException - if fromElement is not compatible with this set's comparator (or, if the set has no comparator, if fromElement does not implement Comparable). Implementations may, but are not required to, throw this exception if fromElement cannot be compared to elements currently in the set.`

`NullPointerException - if fromElement is null and this set does not permit null elements`

`IllegalArgumentException - if this set itself has a restricted range, and fromElement lies outside the bounds of the range`

first

`E first()`

Returns the first (lowest) element currently in this set.

Returns:

`the first (lowest) element currently in this set`

Throws:

`NoSuchElementException - if this set is empty`

last

`E last()`

Returns the last (highest) element currently in this set.

Returns:

`the last (highest) element currently in this set`

Throws:

NoSuchElementException - if this set is empty

spliterator

default Spliterator<E> spliterator()

Creates a Spliterator over the elements in this sorted set.

The Spliterator reports Spliterator.DISTINCT, Spliterator.SORTED and Spliterator.ORDERED. Implementations should document the reporting of additional characteristic values.

The spliterator's comparator (see Spliterator.getComparator()) must be null if the sorted set's comparator (see comparator()) is null. Otherwise, the spliterator's comparator must be the same as or impose the same total ordering as the sorted set's comparator.

Specified by:

spliterator in interface Collection<E>

Specified by:

spliterator in interface Iterable<E>

Specified by:

spliterator in interface Set<E>

Implementation Requirements:

The default implementation creates a *late-binding* spliterator from the sorted set's Iterator. The spliterator inherits the *fail-fast* properties of the set's iterator. The spliterator's comparator is the same as the sorted set's comparator.

The created Spliterator additionally reports Spliterator.SIZED.

Implementation Note:

The created Spliterator additionally reports Spliterator.SUBSIZED.

Returns:

a Spliterator over the elements in this sorted set

Since:

```
1.8
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.